

DSA

Here's all three side by side — the full reference table.

HashMap

Operation	Average Case	Worst Case
<code>put()</code>	$O(1)$	$O(n)$
<code>get()</code>	$O(1)$	$O(n)$
<code>getOrDefault()</code>	$O(1)$	$O(n)$
<code>remove()</code>	$O(1)$	$O(n)$
<code>containsKey()</code>	$O(1)$	$O(n)$
<code>containsValue()</code>	$O(n)$	$O(n)$ — always linear, scans all values
<code>size()</code>	$O(1)$	$O(1)$
<code>keySet()</code> / <code>values()</code> / <code>entrySet()</code> (get the view)	$O(1)$	$O(1)$
Iterating the whole map	$O(n)$	$O(n)$

HashSet

Operation	Average Case	Worst Case
<code>add()</code>	$O(1)$	$O(n)$

<code>contains()</code>	$O(1)$	$O(n)$
<code>remove()</code>	$O(1)$	$O(n)$
<code>size()</code>	$O(1)$	$O(1)$
Iterating the whole set	$O(n)$	$O(n)$

Why HashMap and HashSet match exactly: HashSet is literally implemented internally as a HashMap where every value is a dummy placeholder object — so every HashSet operation is really just calling the equivalent HashMap operation underneath.

PriorityQueue (Heap)

Operation	Time Complexity
<code>offer()</code> / <code>add()</code> (insert)	$O(\log n)$
<code>poll()</code> / <code>remove()</code> (remove root)	$O(\log n)$
<code>peek()</code>	$O(1)$
<code>contains(x)</code>	$O(n)$
<code>remove(x)</code> (specific element, not root)	$O(n)$
Building from a collection (constructor)	$O(n)$
<code>size()</code>	$O(1)$
Iterating (no guaranteed order!)	$O(n)$

Important note on PriorityQueue iteration: iterating a `PriorityQueue` with a for-each loop does **not** give you elements in sorted order — it just walks the internal array as-is. Only repeated `poll()` calls give you elements in min/max order.

Why the complexity classes differ fundamentally

Structure	Underlying mechanism	Core operation cost
HashMap/HashSet	Hashing → direct bucket access	$O(1)$ — no ordering maintained
PriorityQueue	Binary heap → tree structure via array	$O(\log n)$ — maintains min/max ordering

This is the fundamental tradeoff worth internalizing: **HashMap/HashSet are fast because they don't maintain any order. PriorityQueue is slower because it does maintain a partial order (min/max always accessible).** Whenever a problem needs "give me the smallest/largest efficiently, repeatedly, as data changes" — that ordering requirement is exactly why you pay the $O(\log n)$ cost instead of $O(1)$.

One more — since you'll hit it soon in your list: TreeMap/TreeSet

Operation	Time Complexity
<code>put()</code> / <code>add()</code>	$O(\log n)$
<code>get()</code> / <code>contains()</code>	$O(\log n)$
<code>remove()</code>	$O(\log n)$
<code>firstKey()</code> / <code>lastKey()</code> (min/max)	$O(\log n)$

TreeMap/TreeSet maintain **full sorted order** (not just min/max like a heap) — backed by a Red-Black Tree — which is why every operation costs $O(\log n)$, a middle ground between HashMap's $O(1)$ -no-order and needing something fully sorted at all times.

=====

Worst case happens when hashing breaks down — specifically, when many different keys end up landing in the **same bucket**, forcing the HashMap to fall back to a slow, linear search within that bucket instead of the instant direct-access lookup it's designed for.

How HashMap normally works — the "good case"

```
HashMap<Integer, Integer> map = new HashMap<>();
```

Internally, HashMap is just an **array of buckets**. When you `put(key, value)`:

1. Java computes `key.hashCode()` — a number representing that key
2. That hash gets converted into an array index (via something like `hash % arraySize`)
3. The key-value pair gets stored at that specific bucket/index

If every key hashes to a **different** bucket, then `get(key)` is basically: compute hash → jump straight to that array index → done. That's **$O(1)$** — no searching required at all.

Where it breaks down — hash collisions

A **collision** happens when two *different* keys produce the **same** bucket index. When that happens, both key-value pairs need to live in the **same bucket** — and a single bucket internally stores multiple entries as a **linked**

list (or, in Java 8+, a red-black tree once the bucket gets crowded enough — but let's focus on the classic explanation first).

Bucket 5: [key A → valA] → [key B → valB] → [key C → valC]
(all three hashed to the same index, 5)

Now, `get(keyC)` can't jump straight there anymore — it has to:

1. Go to bucket 5
2. Walk through the linked list: check key A (no match) → check key B (no match) → check key C (match!) → return valC

If **all n keys** happened to collide into the **same single bucket** (an extreme, contrived case), you'd essentially have a single giant linked list of n elements, and looking up any key would require potentially scanning **all n** entries — that's **O(n)**, exactly like searching an unsorted linked list.

When would this actually happen in practice?

1. A poorly designed `hashCode()` on a custom object

```
class BadKey {  
    int value;  
    @Override  
    public int hashCode() {  
        return 1; // ALWAYS returns the same hash — guarantees every  
        object collides!  
    }  
}
```

If you use a class like this as a HashMap key, **every single key** lands in the same bucket, no matter how many you insert — worst case guaranteed.

2. A deliberate hash-flooding attack In real-world systems (this is a known security concern), an attacker who understands your hash function can deliberately craft many keys that all hash to the same bucket, intentionally degrading your HashMap's performance from $O(1)$ to $O(n)$ — a denial-of-service technique. This is a real historical vulnerability that's been exploited against web servers using naive hash implementations.

3. Extremely bad luck with default `hashCode()` With Java's default `hashCode()` (usually based on memory address for custom objects, or well-distributed for built-in types like `Integer/String`), this is statistically almost impossible to hit by accident — Java's built-in hash functions for common types are specifically designed to distribute well.

Java 8's mitigation — treeify

If a single bucket accumulates **8 or more** colliding entries, Java automatically converts that bucket's linked list into a **red-black tree**. This caps the worst case for that bucket at **$O(\log n)$** instead of $O(n)$ — a real engineering safeguard against exactly this collision scenario (including intentional hash-flooding attacks), introduced specifically to blunt that security concern.

Why you basically never worry about this in interviews

For built-in key types (`Integer`, `String`, etc.), Java's hash functions are well-tested and well-distributed — collisions are rare and mild in practice, not catastrophic. So in interview contexts, you always state HashMap operations as **$O(1)$ average case**, and only mention the $O(n)/O(\log n)$ worst case if an interviewer specifically probes "what's the worst case, and why?" — which does occasionally come up as a deeper follow-up question to test genuine understanding versus memorization.

=====
=====

Quick recap — here's the full list we covered earlier.

Comparison-based sorts

Algorithm	Best	Average	Worst	Space	Stable?
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	No
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	No
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	No

Non-comparison sorts

Algorithm	Time	Space	Condition
Counting Sort	$O(n + k)$	$O(k)$	Integers in a known small range k
Radix Sort	$O(d \cdot (n + k))$	$O(n + k)$	Fixed-digit-length numbers

Bucket Sort	$O(n + k)$ avg	$O(n)$	Uniformly distributed values / known bounded range
-------------	-------------------	--------	--

Priority for FAANG prep — what to actually know cold

1. **Merge Sort** — code from scratch, understand divide-and-conquer pattern
2. **Quick Sort** — code from scratch, understand partition logic (feeds directly into Quickselect)
3. **Heap Sort** — understand conceptually, know it's what backs `PriorityQueue`-style thinking
4. **Bucket Sort** — know when the "bounded range" signal applies (you just used this for Top K Frequent)
5. **Counting Sort** — same "bounded range" signal, simpler than bucket sort when values ARE the index directly
6. **Insertion Sort** — know Java's `Arrays.sort()` falls back to this for small subarrays

Want to move on to Two Pointers problems now, or is there another sorting-adjacent thing you want to lock in first — like actually coding Quick Sort's partition logic since it's the foundation for Quickselect?

=====

=====

Here's the full breakdown — stability and space, side by side, for every sorting algorithm we've covered.

Complete comparison table

Algorithm	Stable?	In-Place?	Extra Space
m			

Bubble Sort	✓ Yes	✓ Yes	$O(1)$
Selection Sort	✗ No	✓ Yes	$O(1)$
Insertion Sort	✓ Yes	✓ Yes	$O(1)$
Merge Sort	✓ Yes	✗ No	$O(n)$
Quick Sort	✗ No	✓ Yes	$O(\log n)$ — recursion stack only
Heap Sort	✗ No	✓ Yes	$O(1)$
Counting Sort	✓ Yes (if implemented correctly, like we did)	✗ No	$O(n + k)$
Bucket Sort	✓ Yes (depends on how each bucket is sorted internally)	✗ No	$O(n)$
Radix Sort	✓ Yes	✗ No	$O(n + k)$

What "stable" actually means — quick reminder

Two elements that are **equal** (by the sorting key) keep their **original relative order** after sorting. We proved this directly with Radix Sort's backward-processing trick a few messages ago.

What "in-place" actually means

The algorithm sorts using only the original array (plus a small, constant amount of extra space like a few variables) — it does **not** allocate a separate array proportional to the input size.

Why each one is stable or not — quick reasoning per algorithm

Bubble Sort — stable: only swaps **adjacent** elements when the left one is strictly **greater** than the right one. Equal elements never get swapped, so their order never changes.

Selection Sort — unstable: finds the minimum in the unsorted region and swaps it into place — this swap can jump an equal element **past** another equal element that was originally before it, breaking their relative order.

Example: [3a, 1, 3b, 2] (3a and 3b are equal, tracking which was "first")
After selection sort's swaps, 3b might end up before 3a — order not guaranteed.

Insertion Sort — stable: only shifts elements when the next element is strictly **smaller** than what's already in the sorted region — equal elements just stay put, never jump past each other.

Merge Sort — stable: *this is exactly why we used $\leq$ (not $<$) in the merge step*, remember? When two elements are equal, the left one always gets picked first, preserving original order.

Quick Sort — unstable: the partition step swaps elements based on position relative to the pivot, and these swaps can freely reorder equal elements — there's no mechanism protecting their original order.

Heap Sort — unstable: building/maintaining the heap involves swaps based on parent-child comparisons, which can reorder equal elements arbitrarily as they bubble up/down.

Counting Sort — stable (when implemented with the prefix-sum + backward-placement technique, exactly like we did for Radix Sort's digit passes): this is precisely why Radix Sort — which is just repeated Counting Sort — works correctly across multiple passes.

Bucket Sort — stable, IF each bucket preserves insertion order (like ours did, using `ArrayList.add()`) **AND** you don't re-sort each bucket with an unstable algorithm internally. If a bucket needs internal sorting (multiple items with different sub-values in the same bucket) and you use an unstable sort for that, the overall result becomes unstable.

Radix Sort — stable: built directly from stable Counting Sort passes, and we proved this is *necessary* for correctness across multiple digit passes (you actually questioned this yourself, and we worked through why it matters).

Why in-place vs extra-space matters practically

In-place algorithms ($O(1)$ extra space) are preferred when memory is constrained — you're sorting within the given array, not allocating a second array of the same size.

Non-in-place algorithms ($O(n)$ extra space) trade memory for either stability (Merge Sort) or for avoiding comparisons entirely (Counting/Bucket/Radix Sort).

The interview-relevant tradeoff table

Need	Best choice
Stability required + $O(n \log n)$ guaranteed	Merge Sort
Minimal memory, don't care about stability	Quick Sort or Heap Sort

Values in a small known range, need speed	Counting Sort
Need to group actual objects by bounded property	Bucket Sort
Large integers, fixed digit count	Radix Sort

This table is genuinely worth having memorized cold for FAANG interviews — "is this stable, is this in-place" is a very common rapid-fire follow-up right after you finish explaining or coding any sort, especially if you mention using a particular sort as a subroutine in a bigger problem (like we did with Group Anagrams).

=====

=====

Use Counting Sort directly (like this) when your value range *k* is small and manageable — e.g., sorting ages, scores, or small integers.

Use Radix Sort instead when *k* would be enormous (e.g., values up to a billion) — because Radix Sort avoids ever needing a *count* array of size *k*, replacing it with *d* passes of a fixed 10-slot *count* array instead.

Want to trace this through with your own numbers to confirm it clicks the same way Radix Sort did, or are you comfortable moving on now that you've seen how directly these two connect?

=====

=====